CUSTOMER CHURN PREDICTION

End-to-End Machine Learning Pipeline

From raw IBM Telco data to a trained model, evaluation, Streamlit application, and cloud deployment preparation

Project type: Binary classification • **Model:** Logistic Regression • **Interface:** Streamlit

Prepared as a practical project walkthrough

1. End-to-end pipeline at a glance

This project predicts whether a telecom customer is likely to churn (leave the service). The workflow separates data preparation, model selection, final testing, and app deployment so that evaluation is not accidentally biased.

Stage	What happens	Output
1. Load data	Read IBM Telco CSV and inspect shape, columns, and sample rows.	7,043 rows × 21 columns
2. Clean data	Convert TotalCharges to numeric; resolve 11 blank values; check duplicates/nulls.	Clean feature table
3. Explore	Review target balance and churn patterns such as tenure and contract type.	Business observations
4. Define X and y	Remove target and customer ID from predictors; map No/Yes to 0/1.	19 model features
5. Split data	Stratified 60/20/20 split into train, validation, and test.	4,225 / 1,409 / 1,409
6. Preprocess	Scale numeric columns and one-hot encode categorical columns.	45 transformed features
7. Train	Fit Logistic Regression on training data only.	Trained classifier
8. Select threshold	Compare validation performance at 0.3, 0.4, and 0.5.	Chosen threshold: 0.30
9. Test once	Evaluate selected threshold on held-out test set.	Final test metrics
10. Save artifact	Bundle preprocessor, model, threshold, and column lists.	customer_churn_model.joblib
11. Build app	Collect 19 inputs in Streamlit, transform, predict probability, classify risk.	Local Streamlit app
12. Deploy	Push app, model artifact, and requirements to GitHub; deploy through Streamlit Community Cloud.	Shareable app (deployment pending)

Important: cloud deployment was prepared but had not yet been confirmed complete at the time this pipeline was documented.

2. Dataset and business objective

Dataset: IBM Telco Customer Churn, file name `WA_Fn-UseC_-Telco-Customer-Churn.csv`.

• Rows: 7,043 customers.

• Original columns: 21, including customerID and the Churn target.

• Target: Churn = Yes or No.

• Class distribution: No = 5,174 (73.46%); Yes = 1,869 (26.54%).

Business objective: flag customers with a higher estimated probability of leaving, so a retention team can prioritize outreach. A model prediction is a risk signal, not a guarantee that a customer will churn.

3. Load and inspect the raw data

```
import pandas as pd

df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")

print("Shape:", df.shape)
display(df.head())
print(df.info())
```

Expected key check: shape should be (7043, 21). Review column types before doing transformations.

Why this step matters: understand the data structure and identify fields that need cleaning before modeling.

4. Data cleaning

4.1 Convert TotalCharges to numeric

```
df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"],
```

```
    errors="coerce"
)

print("Missing TotalCharges:", df["TotalCharges"].isna().sum())
```

The TotalCharges column was initially stored as text. Converting with errors='coerce' turns blank or invalid entries into missing values rather than silently treating the column as text.

4.2 Resolve the 11 blank values

```
print(df.loc[df["TotalCharges"].isna(), ["tenure", "Churn"]])

df["TotalCharges"] = df["TotalCharges"].fillna(0)

print("Missing values after cleaning:",
      df["TotalCharges"].isna().sum())
```

The 11 blank TotalCharges records had tenure = 0 and Churn = No. Filling TotalCharges with 0 is consistent with customers who have no recorded tenure/charges in this dataset.

4.3 Check duplicates and missing values

```
print("Duplicate rows:", df.duplicated().sum())
print(df.isna().sum().sort_values(ascending=False).head(10))
```

Inspect these outputs before proceeding. Do not drop records or columns automatically without understanding what they represent.

5. Exploratory data analysis

5.1 Target balance

```
print(df["Churn"].value_counts())
print((df["Churn"].value_counts(normalize=True) * 100).round(2))
```

Churn is imbalanced: approximately 73.46% No and 26.54% Yes. A model can achieve deceptively good accuracy by favoring the majority class, so recall, precision, F1, and ROC-AUC are also important.

5.2 Tenure and contract patterns

Observed patterns from the project:

• Non-churn customers had average tenure of about 37.57 months (median 38).

• Churn customers had average tenure of about 17.98 months (median 10).

• Churn rate by contract: Month-to-month 42.71%, One year 11.27%, Two year 2.83%.

These are associations in this dataset, not proof that tenure or contract type alone causes churn.

6. Define features and target

```
X = df.drop(columns=["Churn", "customerID"])
y = df["Churn"].map({"No": 0, "Yes": 1})

print("X shape:", X.shape)
print("y shape:", y.shape)
```

customerID is excluded because it is an identifier rather than a useful customer behavior feature. Churn is the label to predict, so it must not remain inside X.

Feature inventory: 19 predictors: 4 numeric (SeniorCitizen, tenure, MonthlyCharges, TotalCharges) and 15 categorical.

7. Split into train, validation, and test

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.40, stratify=y, random_state=123
)

X_val, X_test, y_val, y_test = train_test_split(
```

```
    X_temp, y_temp, test_size=0.50,
    stratify=y_temp, random_state=123
)

print(len(X_train), len(X_val), len(X_test))
```

Expected row counts: 4,225 training, 1,409 validation, and 1,409 test records. Stratification keeps the churn/non-churn proportion similar across splits.

Why three sets? Training fits the model; validation selects the decision threshold; test provides the final held-out estimate. The test set should not be used to choose the threshold.

8. Build the preprocessing pipeline

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

numerical_columns = [
    "SeniorCitizen", "tenure", "MonthlyCharges", "TotalCharges"
]

categorical_columns = [
    "gender", "Partner", "Dependents", "PhoneService",
    "MultipleLines", "InternetService", "OnlineSecurity",
    "OnlineBackup", "DeviceProtection", "TechSupport",
    "StreamingTV", "StreamingMovies", "Contract",
    "PaperlessBilling", "PaymentMethod"
]

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numerical_columns),
        ("cat", OneHotEncoder(handle_unknown="ignore"),
         categorical_columns)
    ]
)

X_train_processed = preprocessor.fit_transform(X_train)
X_val_processed = preprocessor.transform(X_val)
X_test_processed = preprocessor.transform(X_test)

print(X_train_processed.shape)
```

The training data is used to fit the scaler and encoder. Validation and test data are only transformed. This prevents preprocessing leakage. The transformed feature count in this project was 45.

handle_unknown='ignore' allows the app to process an unseen category without failing at inference time.

9. Train Logistic Regression

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    max_iter=2000,
    random_state=42
)

model.fit(X_train_processed, y_train)

print("Model trained.")
print("Iterations:", model.n_iter_)
```

The model was trained on the training split only and converged in 24 iterations without warnings. Logistic Regression estimates a probability for the positive class, which is useful for threshold-based risk classification.

10. Select a decision threshold using validation data

```
from sklearn.metrics import precision_score, recall_score, f1_score

val_probabilities = model.predict_proba(X_val_processed)[:, 1]

for threshold in [0.3, 0.4, 0.5]:
    val_predictions = (val_probabilities >= threshold).astype(int)
    print(
```

```
        threshold,
        precision_score(y_val, val_predictions),
        recall_score(y_val, val_predictions),
        f1_score(y_val, val_predictions)
    )
```

Validation results from this run:

Threshold	Precision	Recall	F1
0.30	53.89%	75.94%	63.04%
0.40	58.80%	65.24%	61.85%
0.50	64.82%	53.21%	58.44%

Threshold 0.30 was selected to prioritize recall while maintaining the strongest validation F1 among the compared thresholds. A lower threshold catches more potential churners but also creates more false positives.

11. Final evaluation on the held-out test set

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, confusion_matrix
)

selected_threshold = 0.30
test_probabilities = model.predict_proba(X_test_processed)[:, 1]
test_predictions = (test_probabilities >= selected_threshold).astype(int)

print("Accuracy:", accuracy_score(y_test, test_predictions))
print("Precision:", precision_score(y_test, test_predictions))
print("Recall:", recall_score(y_test, test_predictions))
print("F1:", f1_score(y_test, test_predictions))
print("ROC-AUC:", roc_auc_score(y_test, test_probabilities))
print(confusion_matrix(y_test, test_predictions))
```

Final test results at threshold 0.30:

Metric	Test result	Plain-English meaning
Accuracy	76.44%	Overall fraction classified correctly.
Precision	54.04%	Of customers flagged, about 54% actually churned.
Recall	75.13%	About 75% of actual churners were identified.
F1	62.86%	Balance between precision and recall.
ROC-AUC	0.8465	Good ability to rank churners above non-churners across thresholds.

Confusion matrix [[796, 239], [93, 281]] using sklearn's standard layout [[TN, FP], [FN, TP]]. The model correctly identified 281 churners, missed 93 actual churners, and flagged 239 non-churners as high risk.

The held-out test set was kept separate from threshold selection in the final workflow. This is more trustworthy than choosing a threshold based on test performance.

12. Save the complete model artifact

```
import joblib

artifact = {
    "preprocessor": preprocessor,
    "model": model,
    "threshold": selected_threshold,
    "numerical_columns": numerical_columns,
    "categorical_columns": categorical_columns
}

joblib.dump(artifact, "customer_churn_model.joblib")
print("Saved model artifact.")
```

The artifact bundles the preprocessor and classifier together with the threshold and feature lists. The app must use the same preprocessing learned during training; do not refit it on customer input.

13. Streamlit application flow

The local app was built in VS Code in app.py. It collects 19 customer fields, arranges columns in the expected order, transforms the input using the saved preprocessor, predicts churn probability, and compares the probability with the saved 0.30 threshold.

```
# Core inference logic used by app.py
customer_df = pd.DataFrame([customer_input])
customer_df = customer_df[numerical_columns + categorical_columns]

processed_customer = preprocessor.transform(customer_df)
churn_probability = model.predict_proba(processed_customer)[0, 1]

if churn_probability >= selected_threshold:
    st.warning("Higher Churn Risk")
else:
    st.success("Lower Churn Risk")

st.metric("Estimated churn probability",
          f"{churn_probability:.2%}")
```

Example from the local app's default input: estimated churn probability 19.42%, below the 30% threshold, so the app showed Lower Risk. This is one sample prediction, not a model evaluation metric.

13.1 Run locally

```
# Activate the project virtual environment in Windows CMD
.venv\Scripts\activate

# Start the app
python -m streamlit run app.py
```

The project used Python 3.12.13 and scikit-learn 1.6.1 in the local virtual environment. The saved artifact was originally created with scikit-learn 1.6.1.

14. Resolve package compatibility

The first local launch failed with an sklearn serialization error because the global environment used scikit-learn 1.8.0, while the saved artifact came from scikit-learn 1.6.1. Installing 1.6.1 under Python 3.14 failed due to build/toolchain compatibility. The fix was to create a Python 3.12 virtual environment and install the matching scikit-learn version.

```
# requirements.txt
streamlit
pandas
scikit-learn==1.6.1
joblib
```

Pinning the compatible version helps reproduce the model environment locally and in cloud deployment. In a production project, record Python and dependency versions explicitly.

15. GitHub and Streamlit Community Cloud

Repository created: customer-churn-prediction. Files uploaded to the repository root:

• app.py

• customer_churn_model.joblib

• requirements.txt

• README.md — recommended to add/verify before deployment.

Do not upload the .venv folder. It is large and environment-specific; Streamlit Cloud installs dependencies from requirements.txt.

Deployment checklist

• Confirm the GitHub repository is accessible to Streamlit Community Cloud.

• Choose the repository and the correct branch (typically main).

• Set the main file path to app.py.

• Deploy and wait for dependency installation.

• Open the deployed app and test a sample prediction.

• If deployment fails, inspect the app logs first; verify file names, Python/dependency compatibility, and that the model artifact is in the expected path.

Status: local app worked; GitHub files were uploaded; cloud deployment still needed to be completed and the live URL added to README.

16. Final project structure

```
customer-churn-prediction/
|-- app.py
|-- customer_churn_model.joblib
|-- requirements.txt
|-- README.md
```

Keep the notebook or training script separately if you want the full model-building process to be reproducible. The deployed app only needs the saved artifact and inference code.

17. What the project demonstrates

• Data cleaning and type conversion.

• Exploratory analysis of an imbalanced binary target.

• Leakage-aware train/validation/test splitting.

• Preprocessing numeric and categorical features.

• Logistic Regression training and probability-based prediction.

• Threshold selection based on business trade-offs.

• Evaluation using precision, recall, F1, ROC-AUC, and confusion matrix.

• Model packaging and a Streamlit user interface.

• Dependency troubleshooting and deployment preparation.

18. Limitations and next improvements

• The data is a public sample dataset and may not represent a real telecom company's current customer base.

• Synthetic/demo-style app inputs need validation rules and realistic business constraints.

• The 0.30 threshold favors recall, which means a substantial number of non-churners are flagged; retention costs should inform threshold choice.

• No model monitoring, drift detection, retraining process, authentication, or production data integration has been implemented.

• Next: complete cloud deployment, add live URL, add unit tests and input validation, compare models, and evaluate retention campaign value/cost.

19. Final takeaway

This is an end-to-end churn prediction prototype: raw data → cleaning → EDA → leakage-aware split → preprocessing → Logistic Regression → validation-based threshold choice → held-out test evaluation → saved artifact → Streamlit app → GitHub and cloud deployment preparation.

The key lesson is that a useful ML project is more than model accuracy. It includes trustworthy evaluation, a clear business trade-off, reproducible artifacts, and an interface that lets someone use the model.